

과제5_ReactCRUD_과제안내

노트북 기본 정보

항목	내용
과정명	AI SW 장기교육
과제 번호	과제 5
과제명	리액트를 이용해서 CRUD 앱 만들기
연계 특강	특강 5. 프론트엔드에서 백엔드로: 웹 아키텍처 확장과 Supabase MCP 바이브코딩
과제 주제	React 기반 CRUD 앱 구현과 데이터 저장 구조 확장
권장 실행 환경	Cursor, VS Code + Cline, StackBlitz, CodeSandbox, GitHub Codespaces, 로컬 개발 환경 중 선택
기본 저장 방식	mock data 또는 LocalStorage
선택 확장	Supabase DB, Auth/RLS 개념 검토
제출 링크	{{SUBMISSION_LINK}}
최종 제출 마감	과제 응시 10일 후 일요일 23:59, 실제 날짜는 운영 공지 기준

1. 과제 5 전체 구조

1-1. 한 줄 목표

React로 목록 조회, 추가, 수정, 삭제가 가능한 작은 CRUD 앱을 만들고, 구현 전 요구사항·데이터 구조·프롬프트·검토·오류 해결·README 기록까지 완성합니다.

1-2. 과제 산출물 패키지

산출물	설명	필수 여부
React CRUD 앱 소스	목록 조회, 추가, 수정, 삭제 기능을 가진 프로젝트	필수
실행 화면 캡처	목록 화면, 추가/수정/삭제 동작 화면, 빈 상태 또는 오류 상태 화면	필수
README.md	실행 방법, 주요 기능, AI 활용 기록, 오류 해결 기록	필수
프롬프트 기록	요구사항 정리, 파일 구조, 기능 구현, 수정 요청, 오류 해결 프롬프트	필수
AI 생성 결과 검토표	AI가 만든 파일 구조·컴포넌트·기능 코드 검토 기록	필수
오류 해결 기록표	오류가 없으면 “오류 없음”으로 기록	필수
GitHub 또는 Drive 링크	제출용 공유 링크. 공개 범위와 민감 정보 노출 확인	필수
Supabase 설계/보안 기록	Supabase를 선택한 경우 테이블, Auth, RLS, 키 관리 기록	선택

1-3. 과제 수행 흐름

- 과제 요구사항 읽기
- 만들 앱의 주제 선택
- 데이터 구조 정의

- 기능 단위 분해
- 프롬프트 작성
- AI 생성 결과 검토
- 기능별 구현 · 실행
- 오류 메시지 기록 및 수정 요청
- README 작성
- 보안 점검
- 제출 및 10일 보완

1-4. 과제 5에서 만들 수 있는 CRUD 앱 예시

정해진 정답 주제는 없습니다. 아래 중 하나를 선택하거나 본인의 주제로 바꿀 수 있습니다.

예시 주제	데이터 예시	사용 상황
할 일 관리 앱	제목, 완료 여부, 우선순위, 생성일	가장 기본적인 CRUD 연습
학습 기록 앱	학습 주제, 시간, 이해도, 메모	교육 과정 포트폴리오 연계
독서 기록 앱	책 제목, 저자, 상태, 감상	카드 UI 연습
문의 관리 앱	이름 대신 익명 ID, 문의 내용, 처리 상태	상태값·필터 연습
미니 재고 관리 앱	상품명, 수량, 카테고리, 메모	입력 검증 연습
프로젝트 태스크 앱	작업명, 담당 역할, 진행 상태, 마감일	팀 프로젝트 준비

2. 과제 안내 및 응시 방식

2-0. 안내

이 과제는 1시간 안에 완성 코드를 제출하는 시험이 아닙니다. 실시간 응시 시간에는 요구사항, 데이터 구조, 프롬프트, AI 생성 결과 검토, 오류 또는 보완 계획을 반드시 남깁니다. 최종 제출은 10일 보완 기간 동안 기능 완성도와 README 품질을 높여 제출합니다.

메모

실시간 응시 중에는 완성 여부보다 기록 품질을 우선 확인합니다. 특히 초급자는 요구사항표, 데이터 구조표, 1차 프롬프트, 검토표를 남기면 최소 산출물로 인정하고, CRUD 4개 기능 완성은 10일 최종 제출합니다.

2-1. 운영 방식

본 과제는 정답 코드를 따라 입력하는 방식이 아닙니다. 교육생은 **바이브 코딩 도구(AI 네이티브 개발 도구)**를 활용하여 요구사항을 정의하고, AI가 생성한 결과를 검토·수정·실행·기록합니다.

구분	운영 방식	교육생 활동
월요일 특강	선수 학습	React CRUD, 데이터 저장, LocalStorage, Supabase 사고 패턴 이해
목요일 과제 공개	온라인 실시간 접속	1시간 실시간 응시, 요구사항 정리, 1차 구현 또는 README 초안
실시간 응시 후	라이브 피드백	채점 포인트, 흔한 실수, 보완 방향 확인

구분	운영 방식	교육생 활동	
오피스아워	제출 전 1:1 피드백	막힌 부분 질문, 방향성 점검	구현 방
최종 제출	응시 10일 후 일요일 마감	GitHub/캡처/README/기록 제출	최종 차

2-2. 실시간 1시간 응시 권장 시간표

시간	권장 활동	산출물
0~10분	과제 요구사항 읽기, 앱 주제 선택, 수준 선택	요구사항 정리표 초안
10~20분	데이터 구조와 CRUD 기능 분해	데이터 구조표, 기능 분해표
20~35분	파일 구조·컴포넌트 구조 프롬프트 작성 및 AI 생성 요청	프롬프트 1~2개, AI 응답 요약
35~50분	생성 결과 검토, 최소 기능 실행 또는 오류 기록	검토표, 오류 기록표
50~60분	README 초안과 보완 계획 정리	README 초안, 제출 전 보완 계획
이후 60분	공통 피드백 확인	최종 제출 전 보완 체크리스트

2-3. 1시간 실시간 응시와 10일 보완 제출 구조

실시간 1시간 안에 모든 기능을 완성하지 못해도 괜찮습니다. 다만 실시간 응시 시간에는 아래 최소 기록을 반드시 남기고, 10일 보완 기간에 최종 제출물을 완성합니다.

구분	실시간 1시간 최소 산출물	10일 최종 제
요구사항	앱 주제, 사용자, 데이터 구조, 필수 기능 범위	요구사항 보완, 기능 우선순위 확정, 예외 상
프롬프트	요구사항 정리 또는 파일 구조 요청 프롬프트 1~2개	Read/Create/Update/Delete 기능별 프롬:
결과	AI 응답 요약, 생성 결과 검토표, 실행 시도 또는 실행 계획	mock data 또는 LocalStorage 기반 Read/
오류	오류가 있으면 원문 기록, 없으면 “오류 없음” 또는 “보완 예정” 기록	오류 원인·해결·재실행 결과 기록
README	README 목차 초안, AI 활용 기록 일부	실행 방법, 주요 기능, AI 활용, 오류 해결, 보

실시간 1시간 최소 체크리스트

항목	확인
앱 주제와 사용자 정의	☐
데이터 구조표 작성	☐
요구사항 또는 파일 구조 프롬프트 작성	☐
AI 생성 결과 검토표 일부 작성	☐
오류 또는 보완 계획 기록	☐

10일 최종 제출 체크리스트

항목	확인
Read/Create/Update/Delete 4개 CRUD 흐름 확인	☐
README 최종 작성	☐
실행 화면 캡처 추가	☐
AI 활용 기록과 수정 요청 기록 정리	☐
오류 해결 기록 또는 오류 없음 기록	☐
개인정보·API Key·service role key·외부 공유 링크 점검	☐

3. 과제 목표와 수준별 기준

3-1. 공통 목표

1. React로 CRUD 앱의 기본 구조를 설계합니다.
2. 구현 전에 요구사항과 데이터 구조를 먼저 정의합니다.
3. AI 도구에 파일 구조, 컴포넌트 구조, 기능 단위 구현을 단계적으로 요청합니다.
4. AI가 생성한 코드를 검토 기준에 따라 확인합니다.
5. 오류 메시지와 수정 과정을 기록합니다.
6. README에 실행 방법, 주요 기능, AI 활용 기록, 오류 해결 기록을 정리합니다.
7. 선택적으로 LocalStorage 또는 Supabase를 활용해 데이터 저장 구조를 확장합니다.

3-2. 수준별 기준

수준	수행 기준	
초급자	mock data 또는 LocalStorage 기반 CRUD 기능 완주	실시간 1시간에는 요구사항표·데이터 구조표·프롬프트
표준 학습자	컴포넌트 분리, 입력 검증, 빈 상태·오류 상태 처리	필수 기능 전체 동작, 컴포넌트 구조 설명, 예외 처리
심화 학습자	Supabase 연결, Auth/RLS 개념 검토, 보안 주의사항 기록	Supabase 시도 기록, 테이블 구조, RLS/키 관리 주

4. 과제 배경 시나리오

여러분은 작은 업무 또는 학습 상황에서 사용할 수 있는 CRUD 앱 프로토타입을 만들어야 합니다. 중요한 것은 “멋진 완성 코드”가 아니라, 데이터를 어떤 구조로 저장하고 사용자가 어떤 방식으로 추가·수정·삭제하는지 설명 가능한 결과물을 만드는 것입니다.

4-1. 사용자 관점 질문

질문	나의 답변
이 앱은 누구를 위한 것인가요?	
사용자는 어떤 정보를 기록하고 싶나요?	
목록 화면에서 어떤 정보가 보여야 하나요?	
사용자는 어떤 값을 입력하나요?	
수정할 수 있는 값은 무엇인가요?	
삭제 시 사용자가 실수하지 않도록 어떤 확인이 필요한가요?	
데이터가 하나도 없을 때 어떤 안내문을 보여줄까요?	

4-2. 앱 한 줄 소개 작성

이 앱은 [사용자]가 [관리할 정보]를 [조회/추가/수정/삭제]할 수 있도록 돕는 React CRUD 앱

5. 구현 전 요구사항 정리

5-1. 요구사항 정의표

항목	작성 내용
앱 이름	
앱 한 줄 설명	
사용할 기술	React, Vite, CSS, LocalStorage 또는 Supabase 선택
데이터 저장 방식	☐ mock data / ☐ LocalStorage / ☐ Supabase
주요 사용자	
필수 기능	목록 조회, 추가, 수정, 삭제
권장 기능	컴포넌트 분리, 입력 검증, 빈 상태, 오류 상태, 필터/검색 중 선택
도전 기능	Supabase 연결, Auth/RLS 검토, 배포, 테스트 시나리오 확장 중 선택
제외할 기능	결제, 실제 개인정보, 민감 데이터, 과도한 인증 기능 등
제출 파일	GitHub 링크, 실행 캡처, README, 프롬프트 기록, 오류 해결 기록

5-2. 기능 분해표

기능 단위	입력	처리	출력	확인 방법
목록 조회 Read	저장된 데이터	배열을 화면에 렌더링	카드 또는 테이블 목록	새로고침 후 목록 확인
추가 Create	입력 폼 값	새 항목 생성 후 상태 갱신	목록에 새 항목 표시	빈 입력·정상 입력 테스트
수정 Update	기존 항목, 수정 값	특정 id 항목 변경	변경된 값 표시	수정 전후 비교
삭제 Delete	삭제할 항목 id	특정 항목 제거	목록에서 사라짐	삭제 후 개수 확인
저장 Persist	데이터 배열	LocalStorage 또는 Supabase 저장	새로고침 후 유지	새로고침 테스트

6. 데이터 구조 설계

6-1. 기본 데이터 구조 예시

아래는 참고용 구조입니다. 정답 코드가 아니라, 프롬프트 작성 전 사람이 먼저 정리할 데이터 설계 예시입니다.

필드	자료형	필수 여부	설명	예시
id	string 또는 number	필수	항목을 구분하는 고유값	task-001
title	string	필수	목록에 표시할 제목	React 상태 관리 복습
description	string	선택	상세 설명 또는 메모	useState와 배열 업데이트 확인
status	string	선택	상태값	todo, doing, done
createdAt	string	선택	생성 시각	2026-07-09
updatedAt	string	선택	수정 시각	2026-07-09

6-2. 나의 데이터 구조 작성표

필드명	자료형	필수 여부	화면 표시 여부	설명
id		☐	☐	
		☐	☐	
		☐	☐	
		☐	☐	

6-3. 데이터 구조 검토 질문

질문	체크
각 항목을 구분할 id가 있나요?	☐
추가할 때 반드시 필요한 입력값이 명확한가요?	☐
수정할 수 있는 필드와 수정하지 않을 필드가 구분되나요?	☐
삭제할 때 어떤 항목을 삭제하는지 식별할 수 있나요?	☐
LocalStorage 또는 Supabase에 저장해도 문제가 없는 샘플 데이터인가요?	☐
실제 개인정보나 민감 정보가 포함되지 않았나요?	☐

7. 필수 기능, 권장 기능, 도전 기능

7-1. 필수 기능

번호	필수 기능	완료 기준	체크
1	목록 조회 Read	저장된 항목이 카드 또는 테이블 형태로 표시됩니다.	☐
2	항목 추가 Create	입력 폼으로 새 항목을 추가할 수 있습니다.	☐
3	항목 수정 Update	기존 항목의 주요 값을 수정할 수 있습니다.	☐
4	항목 삭제 Delete	선택한 항목을 삭제할 수 있습니다.	☐
5	데이터 구조 정의	구현 전 데이터 필드와 예외 상황을 작성했습니다.	☐
6	AI 활용 기록	사용한 프롬프트와 수정 요청을 README에 기록했습니다.	☐
7	오류 해결 기록	오류 메시지, 원인, 수정 결과를 기록했습니다.	☐

7-2. 권장 기능

번호	권장 기능	완료 기준	체크
1	컴포넌트 분리	App, Form, List, Item/Card 등 역할이 분리됩니다.	☐
2	입력 검증	빈 제목, 너무 긴 입력, 중복 가능성 등을 안내합니다.	☐
3	빈 상태 처리	데이터가 없을 때 “아직 등록된 항목이 없습니다”와 같은 메시지를 표시합니다.	☐
4	오류 상태 처리	저장 실패, 입력 오류, 잘못된 수정 상황을 사용자에게 안내합니다.	☐
5	LocalStorage 저장	새로고침 후에도 데이터가 유지됩니다.	☐
6	검색 또는 필터	상태, 키워드, 카테고리 중 하나로 목록을 좁힙니다.	☐

7-3. 도전 기능

번호	도전 기능	완료 기준	체크
1	Supabase 연결	테이블 구조를 만들고 select/insert/update/delete 중 일부를 연결합니다.	☐
2	Auth 개념 검토	로그인 기능을 실제 구현하지 않더라도 사용자별 데이터 분리 필요성을 설명합니다.	☐

번호	도전 기능	완료 기준	체크
3	RLS 개념 검토	사용자가 자신의 데이터만 볼 수 있어야 하는 이유와 정책 방향을 기록합니다.	☐
4	보안 주의사항 기록	anon key, service role key, <code>.env</code> , GitHub 노출 위험을 README에 기록합니다.	☐
5	배포 또는 공유	실행 링크를 만들고 공개 범위와 민감 정보 노출을 점검합니다.	☐

Supabase는 도전 기능이며 필수 기능이 아닙니다. Supabase를 사용하지 않아도 mock data 또는 LocalStorage 기반 CRUD, 컴포넌트 분리, 입력 검증, 오류 해결 기록, 개선 전후 비교를 충실히 작성하면 충분히 평가받을 수 있습니다. 필수 CRUD 기능이 안정화되기 전에 도전 기능만 먼저 구현하지 않습니다.

8. 사용 가능 도구와 실행 환경

8-1. 권장 도구

도구	사용 목적	주의사항
Google Colab	과제 안내 확인, 프롬프트 작성, 기록표 작성	React 앱 실행은 외부 환경 권장
Cursor	바이브 코딩 기반 React 프로젝트 생성·수정	생성 코드를 그대로 제출하지 않고 검토
VS Code + Cline	대체 AI 코딩 환경	파일 수정 범위 확인
StackBlitz / CodeSandbox	설치 없이 브라우저에서 React 실행	공개 링크에 민감 정보 포함 금지
GitHub Codespaces	클라우드 개발 환경	저장소 공개 범위 확인
로컬 개발 환경	표준 React 프로젝트 실행	Node.js, npm 환경 확인
GitHub	소스 제출 및 버전 기록	API Key, <code>.env</code> , 개인정보 업로드 금지
Supabase Dashboard	선택 도전 기능의 DB, Auth, RLS 확인	service role key 절대 노출 금지

8-2. Vite 기반 React 프로젝트 생성 명령 예시

아래 명령은 실행 환경 안내용입니다. 과제 자료는 완성 코드를 먼저 제공하지 않습니다.

```
npm create vite@latest my-crud-app -- --template react
cd my-crud-app
npm install
npm run dev
```

브라우저 기반 실습을 선택하는 경우 StackBlitz, CodeSandbox, GitHub Codespaces에서 React 템플릿을 사용할 수 있습니다.

9. 바이브 코딩 수행 절차

9-1. 기능별 요청 원칙

한 번에 “CRUD 앱 전체를 만들어줘”라고 요청하지 않습니다. 아래 순서로 나누어 요청합니다.

순서	요청 단위	이유
1	요구사항 검토	내가 만들 앱의 범위를 AI와 확인합니다.
2	데이터 구조 설계	CRUD의 기준이 되는 필드를 먼저 정의합니다.
3	파일 구조 제안	과도한 구조를 피하고 최소 구조를 정합니다.
4	컴포넌트 구조 제안	역할을 나눕니다.
5	Read 구현	목록 표시부터 확인합니다.
6	Create 구현	입력 폼과 상태 갱신을 확인합니다.
7	Update 구현	특정 항목 수정 흐름을 확인합니다.
8	Delete 구현	특정 항목 삭제 흐름을 확인합니다.
9	저장 방식 추가	LocalStorage 또는 Supabase를 선택 적용합니다.
10	검토·수정·README	실행 결과와 과정을 정리합니다.

9-2. 프롬프트 6요소

요소	작성 내용
만들 기능	무엇을 구현할 것인가
사용할 기술	React, Vite, CSS, LocalStorage, Supabase 등
입력값	사용자가 입력하는 값 또는 초기 데이터
출력 형태	화면, 컴포넌트, 파일 구조, 테스트 체크리스트 등
제약 조건	초급자 이해, 파일 분리, 보안, 과제 범위 제한
확인 방법	어떤 화면과 동작으로 성공을 확인할 것인가

10. 프롬프트 작성 공간

아래 프롬프트는 그대로 제출하는 정답이 아니라, 본인의 앱 주제와 데이터 구조에 맞게 수정해 사용하는 작업 지시서입니다. 한 번에 전체 앱을 요청하지 말고 기능 단위로 나누어 요청합니다.

10-1. 요구사항 정리 요청 프롬프트

[프롬프트 박스]

너는 AI SW 장기교육 과제 5의 실습 보조자입니다.
 나는 정답 코드를 따라 치는 것이 아니라, 요구사항을 정의하고 AI 생성 결과를 검토하는 방식

과제명:
 리액트를 이용해서 CRUD 앱 만들기

내가 만들 앱 주제:
 [예: 학습 기록 관리 앱]

요청:
 1. 이 앱의 사용자, 목적, 핵심 기능을 정리해 주세요.
 2. CRUD 기능을 Read/Create/Update/Delete로 나누어 주세요.
 3. 필요한 데이터 필드를 표로 제안해 주세요.

4. 필수 기능, 권장 기능, 도전 기능을 구분해 주세요.
5. 구현 전에 확인해야 할 예외 상황을 알려주세요.
6. 완성 코드를 바로 만들지 말고, 먼저 구현 순서와 검토 기준만 제안해 주세요.

10-2. 파일 구조 요청 프롬프트

[프롬프트 박스]

아래 과제를 구현하기 위한 React 프로젝트 파일 구조를 제안해 주세요.

과제:

React CRUD 앱 만들기

데이터 저장 방식:

[☐ mock data / ☐ LocalStorage / ☐ Supabase 중 선택]

필수 기능:

1. 목록 조회
2. 항목 추가
3. 항목 수정
4. 항목 삭제

요청:

1. 초급자용 최소 파일 구조와 표준 학습자용 컴포넌트 분리 구조를 구분해 주세요.
2. 각 파일의 역할을 한 줄로 설명해 주세요.
3. 어떤 순서로 파일을 만들면 좋은지 알려주세요.
4. GitHub에 올리면 안 되는 파일이나 정보가 있다면 알려주세요.
5. 아직 코드는 작성하지 말고 구조와 이유만 설명해 주세요.

10-3. 컴포넌트 구조 요청 프롬프트

[프롬프트 박스]

React CRUD 앱의 컴포넌트 구조를 설계해 주세요.

앱 주제:

[작성]

데이터 필드:

[작성]

요청:

1. App, 입력 폼, 목록, 항목 카드 또는 행 컴포넌트로 역할을 나눠 주세요.
2. 각 컴포넌트가 받을 props와 담당할 일을 표로 정리해 주세요.
3. 상태는 어디에서 관리하면 좋은지 설명해 주세요.

4. 초급자용 구조와 표준 구조를 구분해 주세요.
5. 코드 작성 전에 확인할 질문 5개를 제시해 주세요.

10-4. Read 기능 구현 요청 프롬프트

[프롬프트 박스]

이제 Read 기능만 먼저 구현하려고 합니다.

현재 상황:

- React + Vite 프로젝트를 사용합니다.
- 데이터는 우선 mock data 배열로 시작합니다.
- 데이터 필드는 다음과 같습니다: [필드 작성]

요청:

1. 목록을 화면에 표시하는 최소 구현을 제안해 주세요.
2. 필요한 파일별 변경 내용을 나누어 주세요.
3. map을 사용할 때 key가 왜 필요한지 설명해 주세요.
4. 데이터가 없을 때 보여줄 빈 상태 메시지를 포함해 주세요.
5. 코드 뒤에는 확인 방법을 체크리스트로 작성해 주세요.
6. 과제 범위를 벗어난 기능은 추가하지 마세요.

10-5. Create 기능 구현 요청 프롬프트

[프롬프트 박스]

이제 Create 기능만 추가하려고 합니다.

현재 동작:

- 목록 조회는 동작합니다.
- 데이터 구조는 다음과 같습니다: [필드 작성]

요청:

1. 입력 폼에서 새 항목을 추가하는 흐름을 구현해 주세요.
2. 빈 제목 또는 필수값 누락 시 안내 메시지를 보여 주세요.
3. 추가 후 입력값이 초기화되도록 해 주세요.
4. 상태 업데이트 방식이 기존 배열을 직접 수정하지 않는지 설명해 주세요.
5. 수정할 파일과 확인 방법을 구분해 주세요.

10-6. Update 기능 구현 요청 프롬프트

[프롬프트 박스]

이제 Update 기능만 추가하려고 합니다.

현재 동작:

- 목록 조회와 항목 추가가 동작합니다.

수정할 수 있는 필드:

[예: title, description, status]

요청:

1. 특정 항목을 선택해 수정 모드로 전환하는 방식을 제안해 주세요.
2. 수정 저장과 취소 흐름을 구분해 주세요.
3. 어떤 id의 항목을 수정하는지 명확히 처리해 주세요.
4. 잘못된 입력이 들어왔을 때의 안내를 포함해 주세요.
5. 수정 후 목록에서 변경 내용이 보이는지 테스트하는 방법을 알려주세요.

10-7. Delete 기능 구현 요청 프롬프트

[프롬프트 박스]

이제 Delete 기능만 추가하려고 합니다.

현재 동작:

- 목록 조회, 항목 추가, 항목 수정이 동작합니다.

요청:

1. 특정 id의 항목만 삭제되도록 구현해 주세요.
2. 삭제 전 확인 절차를 넣을지, 넣지 않을지 장단점을 설명해 주세요.
3. 삭제 후 빈 상태가 될 수 있으므로 빈 상태 메시지를 확인해 주세요.
4. 배열 filter를 사용할 때 어떤 기준으로 삭제되는지 설명해 주세요.
5. 삭제 테스트 체크리스트를 작성해 주세요.

10-8. LocalStorage 저장 요청 프롬프트

[프롬프트 박스]

현재 React CRUD 기능이 mock data 기반으로 동작합니다.

이제 LocalStorage를 사용해 새로고침 후에도 데이터가 유지되도록 확장하려고 합니다.

요청:

1. LocalStorage에 저장할 key 이름을 제안해 주세요.
2. 초기 로딩 시 LocalStorage에서 데이터를 읽는 흐름을 설명해 주세요.
3. 데이터가 바뀔 때 저장하는 흐름을 설명해 주세요.
4. JSON.parse 오류나 저장된 데이터가 비어 있을 때의 처리를 포함해 주세요.
5. 코드 변경 위치와 테스트 방법을 나누어 알려주세요.
6. 개인정보나 민감 데이터는 LocalStorage에 저장하지 않는다는 주의 문구를 README에 넣을

10-9. Supabase 확장 사전 검토 프롬프트

[프롬프트 박스]

React CRUD 앱을 Supabase로 확장하기 전에 설계와 보안 위험을 먼저 검토해 주세요.

앱 주제:

[작성]

데이터 구조:

[필드 작성]

요청:

1. Supabase 테이블 이름과 컬럼 초안을 제안해 주세요.
2. select, insert, update, delete가 각각 어떤 작업인지 설명해 주세요.
3. Auth 없이 공개 CRUD로 만들 때의 위험을 설명해 주세요.
4. Auth를 사용하는 경우 사용자별 데이터 분리가 왜 필요한지 설명해 주세요.
5. RLS가 필요한 이유와 기본 검토 질문을 제시해 주세요.
6. anon key와 service role key의 차이, GitHub 노출 금지 사항을 정리해 주세요.
7. 바로 SQL을 실행하지 말고, 사람이 검토할 수 있는 설계표 형태로 먼저 제안해 주세요.

10-10. AI 생성 결과 검토 요청 프롬프트

[프롬프트 박스]

아래 AI 생성 결과가 과제 5 요구사항을 충족하는지 검토해 주세요.

과제 요구사항:

1. React CRUD 앱
2. 목록 조회, 추가, 수정, 삭제
3. 구현 전 데이터 구조 정의
4. AI 활용 기록과 오류 해결 기록 포함
5. 개인정보, API Key, service role key 노출 금지

AI 생성 결과:

[AI가 만든 파일 구조, 코드, 설명 붙여넣기]

검토 기준:

1. 필수 기능 누락 여부
2. 파일 구조가 과도하게 복잡한지 여부
3. 컴포넌트 역할이 명확한지 여부
4. 상태 업데이트 방식이 안전한지 여부
5. 빈 입력, 빈 목록, 오류 상태 처리가 있는지 여부
6. 보안상 위험한 코드나 키 노출 가능성
7. README에 기록해야 할 내용

출력:

문제점, 수정 우선순위, 수정 요청 프롬프트를 표로 작성해 주세요.

10-11. [코드 셀 자리]

이 공간은 정답 코드를 제공하는 자리가 아니라, AI가 생성한 결과를 검토 기준에 따라 확인한 뒤 붙여넣어 실행하는 자리입니다. API Key, service role key, 토큰, 개인정보, 내부자료는 절대 입력하지 않습니다.

```
# [코드 셀 자리]
# 프롬프트로 생성한 결과를 검토표로 확인한 뒤 필요한 경우에만 실행합니다.
# API Key, service role key, 토큰, 개인정보, 내부자료는 절대 입력하지 않습니다.
```

11. AI 생성 결과 검토표

검토 항목	확인 질문	결과
요구사항 충족	목록 조회, 추가, 수정, 삭제가 모두 포함되었나요?	☐ 예 / ☐ 일부 / ☐ 아니오 ☐
데이터 구조	id와 필수 필드가 명확한가요?	☐ 예 / ☐ 일부 / ☐ 아니오 ☐
파일 구조	파일 역할이 명확하게 분리되었나요?	☐ 예 / ☐ 일부 / ☐ 아니오 ☐
컴포넌트 구조	컴포넌트 역할과 props가 설명 가능한가요?	☐ 예 / ☐ 일부 / ☐ 아니오 ☐
상태 관리	배열 상태를 직접 변경하지 않고 새 배열로 갱신하나요?	☐ 예 / ☐ 일부 / ☐ 아니오 ☐
입력 검증	빈 입력, 잘못된 입력을 처리하나요?	☐ 예 / ☐ 일부 / ☐ 아니오 ☐
빈 상태	데이터가 없을 때 안내 메시지가 있나요?	☐ 예 / ☐ 일부 / ☐ 아니오 ☐
저장 구조	mock data, LocalStorage, Supabase 중 선택이 명확한가요?	☐ 예 / ☐ 일부 / ☐ 아니오 ☐
보안	API Key, service role key, 개인정보가 노출되지 않나요?	☐ 예 / ☐ 일부 / ☐ 아니오 ☐
과도한 구현	필수 CRUD보다 복잡한 인증·DB·상태관리 도구가 먼저 들어가지는 않았나요?	☐ 예 / ☐ 일부 / ☐ 아니오 ☐
README	실행 방법과 AI 활용 기록이 정리되어 있나요?	☐ 예 / ☐ 일부 / ☐ 아니오 ☐

AI 결과 요약 기록

항목	작성 내용
사용한 AI 도구	
입력한 핵심 프롬프트	
AI가 생성한 주요 결과	
그대로 사용하지 않은 부분	
내가 직접 수정한 부분	
추가로 요청한 수정 프롬프트	

✓ 12. 테스트 시나리오와 오류 해결 기록

12-1. 기능 테스트 시나리오

번호	테스트 항목	입력 또는 행동	기대 결과	실제 결과	통과 여부
1	초기 화면	앱 실행	샘플 목록 또는 빈 상태 표시		☐
2	항목 추가	정상 제목 입력 후 추가	목록에 새 항목 표시		☐
3	빈 입력 검증	제목 없이 추가	안내 메시지 표시		☐
4	항목 수정	기존 항목 수정	수정 내용 반영		☐
5	수정 취소	수정 중 취소	기존 값 유지		☐
6	항목 삭제	삭제 버튼 클릭	선택 항목 제거		☐
7	빈 상태	모든 항목 삭제	빈 상태 메시지 표시		☐
8	새로고침	LocalStorage 사용 시 새로고침	데이터 유지		☐
9	Supabase 연결	선택 시 select/insert 테스트	DB와 화면 동기화		☐

12-2. 오류 해결 기본 절차

1. 오류 메시지를 그대로 복사합니다.
2. 어느 환경과 어느 파일에서 발생했는지 적습니다.
3. 바이브 코딩 도구에 오류 해석을 요청합니다.
4. 원인 후보를 2~3개로 좁힙니다.
5. 가장 작은 수정부터 적용합니다.
6. 다시 실행하고 결과를 기록합니다.
7. 같은 오류를 막기 위한 체크리스트를 README에 남깁니다.

12-3. 오류 해결 기록표

번호	발생 상황	오류 메시지	원인 후보	수정 내용	재실행 결과
1					
2					
3					

12-4. 오류 해석 프롬프트

[프롬프트 박스]

아래 오류 메시지를 초급자 기준으로 해석해 주세요.

오류가 발생한 환경:

[Cursor / VS Code / StackBlitz / CodeSandbox / GitHub Codespaces / 로컬 / Supabase]

하려던 작업:

[무엇을 하려 했는지 작성]

오류 메시지:

[오류 메시지를 그대로 붙여넣기]

요청:

1. 오류의 의미

2. 가능한 원인 3가지
3. 확인해야 할 파일 또는 코드 위치
4. 가장 작은 수정 방법
5. 다시 테스트하는 방법
6. 같은 오류를 막는 체크리스트

12-5. [코드 셀 자리]

오류 재현 또는 수정 결과 확인이 필요한 경우에만 사용합니다. 민감 정보가 포함된 코드, Supabase 키, API Key, 개인정보는 붙여넣지 않습니다.

```
# [코드 셀 자리]
# 오류 재현 또는 수정 결과 확인용입니다.
# 민감 정보와 키 값은 절대 입력하지 않습니다.
```

13. 보안·개인정보·저작권·키 관리 안내

13-1. 공통 금지 사항

항목	금지 또는 주의 내용	확인
개인정보	실제 이름, 전화번호, 주소, 주민번호, 개인 이메일 등 사용 금지	☐
민감 데이터	건강, 금융, 법률, 계정 정보, 기관 내부자료 사용 금지	☐
저작권	유료 이미지, 상용 코드, 무단 복제 텍스트 사용 금지	☐
내부자료	회사·기관 내부 문서, 비공개 API, 사내 데이터 업로드 금지	☐
API Key	<code>.env</code> , 토큰, 비밀번호 GitHub 업로드 금지	☐
Supabase service role key	브라우저 코드, README, 캡처, GitHub에 절대 노출 금지	☐
외부 공유 링크	편집 권한 전체 공개, 개인 드라이브 구조 노출 금지	☐
자동화 실행	삭제, 결제, 대량 발송, 권한 변경 자동화 실행 금지	☐

13-2. Supabase 선택 시 추가 점검

점검 항목	확인 질문	체크
키 구분	publishable/anon key와 secret/service role key 차이를 이해했나요?	☐
RLS	공개 schema의 테이블에 RLS 필요성을 검토했나요?	☐
Auth	사용자별 데이터 분리가 필요한지 판단했나요?	☐
권한	누구나 수정/삭제할 수 있는 정책을 만들지 않았나요?	☐
GitHub	<code>.env</code> , 키, 프로젝트 URL이 불필요하게 공개되지 않았나요?	☐
캡처	Supabase 대시보드의 민감 정보가 캡처에 포함되지 않았나요?	☐

14. README 작성 안내

README는 최종 결과물의 사용 설명서이자 과제 수행 과정 기록입니다. 기능이 일부 부족하더라도 요구사항, AI 활용, 오류 해결, 보완 계획을 충실히 기록해야 합니다.

메모

README는 기능 완성도뿐 아니라 학습 과정과 검토 태도를 확인하는 핵심 제출물입니다. 실행 방법, AI 활용 기록, 오류 해결 기록, 보안 점검 누락 여부를 우선 확인합니다.

최종 제출물에는 README가 반드시 포함되어야 합니다. README는 단순 설명 문서가 아니라 “무엇을 만들었는지, 어떻게 만들었는지, AI를 어떻게 활용하고 검토했는지, 어떤 오류를 어떻게 해결했는지”를 보여주는 학습 기록입니다.

README 필수 포함 항목

항목	포함 여부
과제명과 한 줄 소개	☐
실행 화면 캡처 또는 캡처 링크	☐
주요 기능 목록	☐
사용 기술	☐
실행 방법	☐
폴더/파일 구조	☐
데이터 구조	☐
AI 활용 기록	☐
오류 해결 기록	☐
테스트 기록	☐
개인정보·API Key·저작권 점검 결과	☐
배운 점과 보완할 점	☐

README 작성에는 별도 파일 `과제5_README_템플릿.md`를 사용합니다.

15. 제출물 체크리스트

실시간 응시 직후에는 최소 기록 중심으로 정리하고, 최종 제출 전에는 실행 가능한 결과물과 README를 함께 점검합니다.

메모

제출 링크 접근 권한, README 누락, 캡처 누락, 키 노출 여부를 우선 점검합니다. 보완 대상자는 기능보다 기록 누락 여부를 먼저 확인합니다.

제출물	설명	확인
GitHub 또는 Drive 링크	소스 코드 저장소 또는 결과물 폴더 링크	☐
실행 화면 캡처	목록, 추가, 수정, 삭제, 빈 상태 또는 오류 상태	☐
README.md	실행 방법, 기능 설명, AI 활용 기록, 오류 해결 기록 포함	☐
프롬프트 기록	요구사항, 파일 구조, 기능 구현, 수정 요청, 오류 해결 프롬프트	☐

제출물	설명	확인
AI 생성 결과 검토표	AI 결과를 어떤 기준으로 확인했는지 기록	☐
오류 해결 기록표	오류가 없으면 “오류 없음”으로 작성	☐
보안 점검	API Key, service role key, 개인정보, 내부자료 노출 없음	☐
저작권 점검	외부 이미지·코드·텍스트 사용 출처 또는 대체 자료 확인	☐
외부 공유 링크 점검	링크 접근 권한, 편집 권한, 민감 정보 노출 여부 확인	☐

제출 전 자기 점검 문장

본인은 AI가 생성한 결과를 그대로 제출하지 않고, 과제 요구사항에 맞게 검토·수정하였습니다. 또한 개인정보, 민감 데이터, 저작권, 내부자료, API Key, service role key, 외부 공유 링크 위험을 점검하였습니다.

16. 평가 기준 요약

평가 항목	확인 내용
A. 필수 CRUD 기능 완성도	목록 조회, 추가, 수정, 삭제 기능이 요구사항대로 동작하는가
B. 요구사항 정의와 데이터 구조	구현 전 사용자, 입력, 처리, 출력, 데이터 필드, 예외 상황을 정리했는가
C. AI 활용 과정과 프롬프트 품질	기능별 프롬프트, AI 응답 검토, 수정 요청 과정이 기록되었는가
D. 결과 검토와 오류 해결	검토표, 오류 메시지, 원인 분석, 수정 기록, 재실행 결과가 남아 있는가
E. README와 제출 품질	실행 방법, 기능 설명, 캡처, AI 활용 기록, 보안 점검이 포함되었는가
F. 권장·도전 기능 또는 개선 태도	LocalStorage, 컴포넌트 분리, Supabase, 테스트, 개선 전후 비교를 시도했는가

Supabase는 선택 도전 기능이므로 Supabase 미사용 자체는 감점 사유가 아닙니다.

17. 보완 제출 안내

보완 제출은 단순히 빠진 코드를 추가하는 것이 아니라, 어떤 기준에서 부족했고 어떻게 개선했는지 설명하는 과정입니다.

보완 제출 기록표

항목	작성 내용
보완 전 부족했던 점	
피드백 내용 요약	
수정한 파일 또는 기능	
수정 전후 차이	
재실행 결과	
README 업데이트 여부	☐ 완료 / ☐ 미완료
보안 재점검 여부	☐ 완료 / ☐ 미완료

피드백 반영 프롬프트

[프롬프트 박스]

아래 피드백을 반영하여 React CRUD 과제 결과물을 보완하려고 합니다.

피드백 내용:

[피드백 붙여넣기]

현재 결과물 상태:

[현재 상태 작성]

요청 사항:

1. 피드백을 기능 단위로 나눠 주세요.
2. 반드시 수정해야 할 항목과 선택 보완 항목을 구분해 주세요.
3. 수정할 파일과 확인 방법을 제안해 주세요.
4. README에 어떤 내용을 추가하면 좋을지 알려주세요.
5. Supabase 또는 API Key 관련 보안 위험이 있다면 제외해 주세요.